

Assignment

NAME:- RUMMAN HAMID Roll #:- 93
Class:- BS-Computer Science Evening (3rd Sem)
Subject:- Computer Organization & Assembly Language
Course Code:- DC-221

Question

Write an Assembly language program for sorting the numbers by using bubble sorting technique.

The sorting should be one of the four types that are given:-

- | | |
|--------------------------|------------------------|
| 1- Unsigned Incrementing | 2- Signed Incrementing |
| 3- Unsigned Decrementing | 4- Signed Decrementing |

Type of sort and data should be passed by argument to the bubble sort subroutine.

Solution

```
[org 0x0100]
```

```
jmp start
```

```
data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0
```

```
data1: dw -60, 45, -50, -55, 35, 10, -30, 10, 0, -25
```

```
swap: db 0
```

```
start:
```

```
    mov bx, 0
```

```
    mov byte[swap], 0
```

```
; Passing Arguments to bubble sort subroutine
```

```
    mov si, 1
```

```
    mov di, data
```

```
    call bubble_sort
```

```
    mov ax, 0x4c00
```

```
    int 0x21
```

bubble_sort:

push bx

push cx

push dx

push si

mov bx, di

mov cx, 10

outer_loop:

xor dx, dx

mov si, bx

inner_loop:

mov ax, [si]

mov dx, [si+2]

; Compare and Swap based on the type of sort

mov al, byte [si+4]

```
cmp al, 1
je unsigned_inc
cmp al, 2
je signed_inc
cmp al, 3
je unsigned_dec
cmp al, 4
je signed_dec
```

unsigned_inc:

; For Unsigned increment sorting

```
cmp ax, dx
```

```
jae skip_swap
```

```
jmp jmp swap_elements
```

~~unsigned_dec:~~

;

signed_inc:

; For Signed Increment Sorting

cmp ax, dx

jge skip_swap

jmp swap_elements

unsigned_dec:

; For Unsigned Decrement Sorting

cmp ax, dx

jbe skip_swap

jmp swap_elements

signed_dec:

; For Signed Decrement Sorting

cmp ax, dx

jl skip_swap

jmp swap_elements

skip_swap:

add si, 2

dec cx

jnz inner_loop

; Check if a Swap has been done

cmp dx, 0

je end_sort

jmp outer_loop

swap_elements:

; Swap the numbers

xchg ax, dx

mov [si], ax

mov [si+2], dx

; Set swap flag to indicate a swap has been done

mov byte [swap], 1

jmp inner loop

end_sort:

pop si

pop dx

pop cx

pop bx

ret
